Ethereum

Etherium

Ethereum Blockchain Technology is a decentralized, open-source blockchain platform that enables developers to build and deploy **smart contracts** and **decentralized applications (dApps)**.

It allows anyone to write and deploy code (called smart contracts) that runs exactly as programmed, without any possibility of fraud, censorship, or third-party interference.

It was proposed in **2013** by **Vitalik Buterin** and launched in **2015**.

Ethereum Features

Smart Contracts – Self-executing agreements with predefined rules written in code.
Example: A crowdfunding contract automatically refunds contributors if the goal isn't met.

Ether (ETH) – The native cryptocurrency used to pay for transactions and computational services on the network.

Decentralized Applications (dApps) – Applications that run on the Ethereum network instead of a single centralized server.

Ethereum Virtual Machine (EVM) – The runtime environment that executes smart contracts.

Consensus Mechanism – Ethereum uses Proof of Stake (PoS) (after the "Merge" update) to validate transactions and secure the network.

Public Ledger – All transactions are recorded in a transparent and immutable blockchain.

Challenges in Implementing Smart Contracts

Nontechnical Parties

Smart contracts are written in programming languages (e.g., Solidity). Nontechnical people (business owners, lawyers, customers) often cannot understand or verify the code. This creates communication gaps and dependence on technical experts.

Example:

A landlord and a tenant agree on rent payments through a smart contract. The landlord is not a programmer and cannot read Solidity code. If the developer encodes a wrong term—e.g., rent due every *15 days* instead of *30 days*—the landlord may not notice until the contract executes automatically.

Challenges in Implementing Smart Contracts

Off-Chain Resources

Smart contracts operate on-chain, but they often need real-world data such as prices, weather conditions, or shipment status. Since blockchains cannot fetch external data directly, they rely on **oracles**, which may introduce errors or trust issues.

Example:

A crop insurance contract pays farmers automatically if rainfall is below a certain level. The contract relies on a weather oracle. If the oracle reports incorrect rainfall data due to a technical glitch, farmers may either get **unjust payments** or **be denied legitimate claims**.

Challenges in Implementing Smart Contracts

Final Agreement Uncertainty (Text vs. Code Conflict)

Traditional contracts use written text. Smart contracts use code. If the written agreement and the code encode different rules, it becomes unclear which one has legal authority.

Example:

A written agreement says payment will be released after **10 days**, but the smart contract code releases it after **7 days**. If the buyer claims the release was premature, disputes arise:

Should the court follow the written text or the executed code?

Challenges in Implementing Smart Contracts

Automation Conflicts with Real Business Practices

Smart contracts execute automatically and strictly. Real business situations often allow flexibility, negotiation, or forgiveness.

Example:

A supplier is late by 1 hour due to a transportation issue. A normal contract might allow leniency. But a **smart contract** may automatically apply a penalty or cancel the agreement, even though both parties would have preferred flexibility.

Challenges in Implementing Smart Contracts

Modifying and Terminating Smart Contracts

Blockchains are immutable. Once deployed, smart contracts are very difficult or expensive to change or stop.

Example:

Two companies sign a smart contract for long-term service. Later they want to modify the service fee. However, the contract is immutable—they must either:

deploy a new contract, or use a complex upgrade pattern. Both options are costly and technically challenging.

Challenges in Implementing Smart Contracts

Objectivity vs. Ambiguity

Smart contracts demand precise, objective conditions. Traditional contracts deliberately use flexible or subjective terms like “reasonable time” or “best effort.”

Example:

A service contract says the provider must respond in a “reasonable time.” Smart contracts cannot encode “reasonable”—they need a fixed number, like **24 hours.** This removes the natural flexibility of human agreements.

Challenges in Implementing Smart Contracts

Payment Guarantees

Smart contracts execute payments only if the digital wallet has enough funds.

If the user forgets to deposit funds in the wallet, the contract fails.

Example:

A smart contract is supposed to pay freelancers automatically when work is completed. But the employer's wallet has insufficient funds on the payment date.

Result: The contract fails, and payment is not transferred—causing delays and disputes.

Challenges in Implementing Smart Contracts

Allocation of Risk

Smart contracts introduce new types of risks:

coding mistakes, vulnerabilities, hacks, oracle failures, gas price spikes

Parties must decide who bears these risks.

Example:

A smart contract has a minor coding bug that locks funds permanently.

Who takes responsibility?

the business?

the smart contract developer?

both parties?

Without clear terms, conflicts arise.

Challenges in Implementing Smart Contracts

Governing Law and Venue

Smart contracts operate across global blockchains, but legal systems are territorial. Parties must specify:

- which country's laws apply
- where disputes will be settled

Example:

A Bangladeshi buyer and a German seller use a smart contract. Dispute arises:

- Which country’s law applies?
- Where should the case be filed—Dhaka or Berlin?
- Without an agreed clause, international disputes become complicated.

Challenges in Implementing Smart Contracts

Best Practices (Hybrid Approach & Legal Clarity)

Smart contracts are still evolving. Reliable standards and legal frameworks are not yet fully mature. Best practice today is a **hybrid model**:

- The legal text describes the agreement.
- The smart contract executes certain parts.

Example:

A commercial lease contract might use:

- Text contract for obligations, dispute resolution, governing law
- Smart contract for automated rent payments

This combination keeps automation while preserving legal clarity.

Oracles

An **oracle** in Ethereum is a service or mechanism that feeds external, off-chain data into smart contracts, which otherwise have no way to access anything outside the blockchain itself.

Why oracles are needed

Smart contracts run in a deterministic, isolated environment (the EVM). They can only see data that's already on-chain — they can't natively make an HTTP request, check a stock price, read the weather, or know the outcome of a football match. But many real-world use cases (insurance, prediction markets, DeFi lending, derivatives) depend on exactly this kind of external information. Oracles bridge that gap.

Oracles

How they work (basic flow)

- A smart contract requests data (e.g., "what's the current ETH/USD price?").
- An oracle service fetches this data from one or more off-chain sources (APIs, exchanges, IoT devices, etc.).
- The oracle submits the data on-chain via a transaction.
- The smart contract consumes that data to execute its logic (e.g., trigger a liquidation, settle a bet, release funds).

The "Oracle Problem"

Since the blockchain itself can't verify whether off-chain data is accurate, oracles introduce a point of centralization and trust — if a single oracle is malicious or compromised, it can feed false data and manipulate contract outcomes. This is known as the **oracle problem**, and it's a major security concern in **DeFi** (flash loan attacks often exploit manipulated price oracles).

Types of Blockchain oracles

There are various types of blockchain oracles, ranging from simple software oracles to complex hard ware-assisted and decentralized oracles.

We can categorize oracles into two categories:

- ❑ Inbound oracles
- ❑ Outbound oracles.

Inbound oracles

This class represents oracles that receive incoming data from external services and feed it into the smart contract.

- ❖ Software oracles
- ❖ Hardware oracles
- ❖ Computation oracles
- ❖ Aggregation-based oracles
- ❖ Crowd wisdom-driven oracles
- ❖ Decentralized oracles
- ❖ Smart oracles

Types of Blockchain oracles

Software oracles

These oracles are responsible for acquiring information from online services on the internet. This type of oracle is usually used to source data such as weather information, financial data (stock prices, for example), travel information, and other types of data from third-party providers.

The data source can also be an internal enterprise system, which may provide some enterprise-specific data. These types of oracles can also be called standard or simple oracles.

A problem with these oracles is that as they are based simply on an online source of data, such as a website; if the source is not live anymore or has failed for some reason, then the feed to the blockchain smart contract will fail.

Types of Blockchain oracles

Illustrative Example

Scenario

Imagine a **crop insurance smart contract** for farmers.

How it works

The contract checks rainfall data.

If rainfall is **less than 10mm for 10 days**, it automatically **pays compensation** to the farmer.

Where the data comes from

The smart contract does **not** know rainfall data by itself.
It uses a software oracle that pulls data from a website.

Types of Blockchain oracles

What goes wrong

Now suppose:

the weather website **shuts down**, or

the server **crashes**, or

the API **stops responding**

Then:

The oracle **cannot send rainfall data** to the blockchain

The smart contract **cannot check its condition,** Even if rainfall was actually low,
no compensation is triggered, So the system fails **not because of the blockchain**, but because the **data source failed**.

Types of Blockchain oracles

Hardware oracles

This type of oracle is used to source data from hardware sources such as IoT devices or sensors. This is useful in use cases such as insurance-related smart contracts where telemetry sensors provide certain information, for example, vehicle speed and location. This information can be fed into the smart contract dealing with insurance claims and payouts to decide whether to accept a claim or not. Based on the information received from the source hardware sensors, the smart contract can decide whether to accept or reject the claim.

These oracles are useful in any situation where real-world data from physical devices is required. However, this approach requires a mechanism in which hardware devices are tamper-proof or tam per-resistant. This level of security can be achieved by providing cryptographic evidence (non-repudiation and integrity) of an IoT device's data and an anti-tampering mechanism on the IoT device, which renders it useless if there are any tampering attempts.

Types of Blockchain oracles

Computation oracles

These oracles allow computing-intensive calculations to be performed off-chain. As blockchain is not suitable for performing compute-intensive operations, a blockchain (that is, a smart contract on a blockchain) can request computations to be performed on off-chain, high-performance computing infrastructure and get the verified results back via an oracle.

Some blockchain tasks require **heavy computation** (e.g., large mathematical calculations, simulations, video processing). But blockchains like Ethereum are **not designed** for this because:

- computation is **slow and expensive**
- every node would need to repeat the same calculation
- gas costs would be extremely high

So instead, the smart contract can **send the task off-chain** to a powerful computing system. An **oracle** then returns the result **with proof** that it is correct.

Truebit is an example of such an oracle-based system.

Types of Blockchain oracles

Aggregation-based oracles

In this scenario, a single value is sourced from many different feeds. As an example, this single value can be the price of a financial instrument, and it can be risky to rely upon only one feed. To mitigate this problem, multiple data providers can be used where all of these feeds are inspected, and finally, the price value that is reported by most of the feeds can be picked up. The assumption here is that if the majority of the sources report the same price value, then it is likely to be correct. The collation mechanism depends on the use case: sometimes it's merely an average of multiple values, sometimes a median is taken of all the values, sometimes it's a mean value, and sometimes it's the maximum value.

Care must be taken here because a single malicious node can skew the results of a mean value by reporting a very large or small number. Usually, median values are used in practice because if most nodes are honest, the median value can be reasonably trusted.

Types of Blockchain oracles

Crowd wisdom-driven oracles

This is another way that the blockchain oracle problem can be addressed where a single source is not trusted. Instead, multiple public sources are used to eventually deduce the most appropriate data. It solves the problem where a single source of data may not be as trustworthy or accurate as expected. If there is only one source of data, it can be unreliable and risky to rely on entirely. It may turn malicious or become genuinely faulty. In this case, to ensure the credibility of data provided by third-party sources for oracles, data is sourced from multiple sources. These sources can be users of the system or even members of the general public who have access to and have knowledge of some data, for example, a political event or a sporting event where members of the public know the results and can provide the required data. Similarly, this data can be sourced from multiple different news websites.

Types of Blockchain oracles

Decentralized oracles

Another type of oracle, which primarily emerged due to decentralization requirements, is a decentralized oracle. As blockchain platforms such as Bitcoin and Ethereum are fully decentralized, it is expected that oracle services should also be decentralized.

This type of oracle can be built based on a distributed mechanism. It can also be envisaged that the oracles can find themselves source data from another blockchain, which is driven by a distributed consensus, thus ensuring the authenticity of data. For example, one institution running its private blockchain can publish its data feed via an oracle that can then be consumed by other blockchains.

Types of Blockchain oracles

Outbound oracles

This type, also called a reverse oracle, is used to send data out from the blockchain smart contracts to the outside world. There are two possible scenarios here; one is where the source blockchain is a producer of some data such as blockchain metrics, which are needed for some other blockchain. The actual data somehow needs to be sent out to another blockchain smart contract.

The other scenario is that an external hardware device needs to perform some physical activity in response to a transaction on-chain.

Etherium Blockchain

A list of elements present in the Ethereum blockchain is presented here:

- Ether cryptocurrency/tokens
- Keys and addresses
- Accounts
- Transactions and messages
- The EVM
- The Ethereum blockchain
- Nodes and miners
- The Ethereum network
- Smart contracts and native contracts
- Wallets and client software

Cryptocurrency

Ether (ETH) – The Native Currency of Ethereum

- **Ether (ETH)** is the native cryptocurrency of the Ethereum blockchain.
- Originally, **miners** received ETH as a reward for:
  - Verifying transactions
  - Validating blocks
  - Securing the Ethereum network
- ETH also serves as the **fuel for computation** on Ethereum.

Ethereum and the DAO Hack

- In **2016**, the **Decentralized Autonomous Organization (DAO)** was hacked.
- The incident led to a major debate within the Ethereum community.
- A **hard fork** was proposed to mitigate the consequences of the hack.
- The hard fork resulted in **two separate blockchains**:
  - **Ethereum (ETH)** – the forked chain
  - **Ethereum Classic (ETC)** – the original chain

Keys and Addresses

Keys and Addresses in Ethereum

- Keys and addresses are fundamental to **ownership and transfer of Ether (ETH).**
- Ethereum uses a **public-key cryptographic system**.
- Each account is associated with:
  - **Private Key** → kept secret
  - **Public Key** → derived from private key
  - **Address** → derived from public key
- **Basic Relationship**

  Private Key → Public Key → Ethereum Address

Keys and Addresses

What is a Private Key?

- A private key is a **randomly generated 256-bit positive integer**.
- It must remain **secret**.
- Whoever controls the private key can control the associated Ethereum account.
- Ethereum uses the **secp256k1 elliptic curve**.

Valid Range

$1 \leq \text{PrivateKey} < n$

where **n** is the order of the secp256k1 curve.

Example

b51928c22782e97cca95c490eb958b06fab7a70b9512c38c36974f47b954ffc4

256 bits

64 hexadecimal characters

Keys and Addresses

What is a Public Key?

The public key is mathematically derived from the private key.

It is a **point on the secp256k1 elliptic curve**.

The point consists of two coordinates:

PublicKey=(x,y) Unlike the private key, the public key does **not need to be kept secret**.

Ethereum Public Key Format

An uncompressed Ethereum public key contains:

X coordinate: 32 bytes

Y coordinate: 32 bytes

Prefix: 1 byte (04)

Therefore:

1+32+32=65 bytes=130 Hex characters

Keys and Addresses

From Public Key to Ethereum Address

An Ethereum address is **not the complete public key**.

Instead:

Step 1

Take the public key.

Step 2

Apply **Keccak-256** hashing.

Step 3

Take the **rightmost 20 bytes (160 bits)** of the resulting hash.

Step 4

Represent those 20 bytes in hexadecimal with 0x prefix.

Formula

Address=Last 20 bytes(Keccak-256(PublicKey))

Keys and Addresses

Example of Address Generation

Suppose the public key is:

3aa5b8eefd12bdc2d26f1ae348e5f383...

Apply Keccak-256

Keccak-256(Public Key)

↓

32-byte hash

Suppose the resulting hash ends with:

77b4b5699827c5c49f73bd16fd5ce3d828c36f32

Take the **last 20 bytes**:

77b4b5699827c5c49f73bd16fd5ce3d828c36f32

Add 0x:

0x77b4b5699827c5c49f73bd16fd5ce3d828c36f32

This becomes the Ethereum address.

Keys and Addresses

Characteristics of an Ethereum Address

Ethereum Address

0x77b4b5699827c5c49f73bd16fd5ce3d828c36f32

20 bytes

160 bits

- Normally represented by **40 hexadecimal characters**
- **0x** is a prefix indicating hexadecimal notation.
- Used to identify an Ethereum account.

Important

An address can be **shared publicly**.

Keys and Addresses

Private Key vs Public Key vs Address

Feature	Private Key	Public Key	Address
Purpose	Sign transactions	Verify/signature relationship	Identify account
Size	256 bits	512-bit coordinates + prefix	160 bits
Representation	64 hex characters	130 hex characters	40 hex characters
Secret?	**Yes**	No	No
Derived from	Random generation	Private key	Public key
Example	`b519...ffc4`	`04...`	`0x77b4...6f32`

Ethereum Accounts

What is an Account?

- An **account** is one of the fundamental building blocks of Ethereum.
- Accounts allow users and smart contracts to **interact with the blockchain**.
- An account has:
  - An **address**
  - A **state**
  - A **balance**
  - A **nonce**
- Transactions cause changes to the state of accounts.
- The collection of all account states represents the **state of the Ethereum network**.
- **Basic Concept**
- **Account → Transaction → State Transition → New Blockchain State**

Ethereum as a State Machine

Ethereum as a State Machine

Ethereum can be viewed as a **transaction-driven state machine**.

State

The Ethereum state contains information about **all accounts**.

Let:

St=Current Ethereum State

A transaction causes a transition:

St+1=F(St,T)

where:

St = previous state

T = transaction

F = Ethereum State Transition Function

St+1 = resulting state

Ethereum as a State Machine

Ethereum State Transition Function

The **State Transition Function** determines how Ethereum processes a transaction.

Main Steps

- ❖ Validate the transaction
- ❖ Check balance and nonce
- ❖ Execute the transaction
- ❖ Handle failures and gas
- ❖ Finalize the resulting state

Ethereum as a State Machine

Ethereum State Transition Function: An illustrative Example

The **Ethereum State Transition Function (STF)** describes how Ethereum transforms the current blockchain state into a **new state** when a valid transaction is executed.

We can express it conceptually as:

$S_{new}=F(S_{old},T)$

where:

S_{old} = current Ethereum state

T = transaction

F = State Transition Function

S_{new} = resulting state

Let's understand the five steps using **one continuous example**.

Ethereum as a State Machine

Example:

Suppose Alice wants to send **2 ETH** to Bob.

Before the transaction

Account	Balance	Nonce
Alice	10 ETH	5
Bob	3 ETH	12

Alice creates a transaction:

```
From:       Alice
To:         Bob
Value:      2 ETH
Nonce:      5
Gas Limit:  21,000
Gas Price:  20 gwei
```

Now Ethereum processes this transaction through the State Transition Function.

Ethereum as a State Machine

❑ Validate the Transaction

Before changing anything, Ethereum first checks whether the transaction is **valid**.

What is checked?

❖ Transaction format

Ethereum checks whether the transaction contains valid fields such as:

- Nonce
- To
- Value
- Gas Limit
- Fee-related fields
- Signature

Ethereum as a State Machine

❖ **Digital signature**

Alice signs the transaction using her **private key**.

Alice's Private Key

↓

Digital Signature

↓

Transaction

Ethereum verifies the signature using cryptographic mechanisms associated with Alice's account.

If the signature is invalid:

❌ Transaction is rejected.

Ethereum as a State Machine

❖ **Nonce**

Alice's current nonce is:

Alice's nonce = 5

The transaction must use the appropriate next nonce.

If the transaction contains an invalid nonce, Ethereum will not execute it as a normal valid transaction.

Example

Transaction nonce = 5

Alice's current expected nonce = 5

↓

VALID ✓

Result

The transaction passes the basic validity checks and can proceed.

Ethereum as a State Machine

❑ Check Balance and Nonce

Now Ethereum determines whether Alice has enough ETH to cover the transaction's requirements.

Alice wants to send:

2 ETH

She also needs to pay the transaction fee.

For a simple example, suppose the fee is:

0.00042 ETH

Therefore, Alice needs at least:

2+0.00042=2.00042 ETH

Alice has:

10 ETH

Therefore:

10>2.00042

Alice has sufficient funds ✓

Ethereum can proceed.

If Alice had only 1 ETH?

She would need:

`Required = 2.00042 ETH`

`Available = 1 ETH`

Therefore:

1<2.00042

❌ **Transaction cannot be successfully executed.**

Ethereum as a State Machine

❖ **Nonce**

Alice's current nonce is:

Alice's nonce = 5

The transaction must use the appropriate next nonce.

If the transaction contains an invalid nonce, Ethereum will not execute it as a normal valid transaction.

Example

Transaction nonce = 5

Alice's current expected nonce = 5

↓

VALID ✓

Result

The transaction passes the basic validity checks and can proceed.

Ethereum Account

An **Ethereum account** is an entity on the Ethereum blockchain that can **hold ETH, hold or interact with tokens, send transactions, and—depending on the account type—execute smart-contract code**.

For example:

Ethereum Blockchain

```
    ├── Alice's Account
    │     Address: 0xABC...
    │     ETH Balance: 10 ETH
    ├── Bob's Account
    │     Address: 0xDEF...
    │     ETH Balance: 5 ETH
    └── USDT Contract Account
        Address: 0x123...
        Code: USDT smart-contract code
        Storage: USDT balances
```

Ethereum Account

Externally owned account (EOA):

EOAs are similar to accounts in Bitcoin that are controlled by a pri vate key.

Defined by three elements: address, balance, and nonce.

They have the following properties:

- They have a state.
- They are associated with a human user, hence are also called user accounts
- EOAs have an ether balance.
- They can send transactions, i.e., can transfer value and can initiate contract code.
- They have no associated code.
- They are controlled by private keys.
- An EOA’s public address is derived from its private key.
- EOAs cannot initiate a call message.
- Accounts contain a key-value store.
- EOAs can initiate transaction messages.

Ethereum Account

Contract account (CA):

CAs are accounts that have code associated with them in addition to being controlled with the private key. They are defined by five elements: address, balance, nonce, StorageRoot, and codeHash.

- They have the following properties:
  - They have a state.
  - They are not intrinsically associated with any user or actor on the blockchain.
  - CAs have an ether balance.
  - They have associated code that is kept in memory/storage on the blockchain. They can get triggered and execute code in response to a transaction or a message from other contracts. A CA's public address is the combination of the public address of the EOA that created it and a transaction counter (how many transactions the EOA has sent) nonce.

Ethereum Account

- They have access to storage and can manipulate their storage.
- Also, CAs can maintain their permanent states and can call other contracts
- A CA can only be created from an EOA or from an already existing CA (contract).
- CAs cannot start transaction messages.
- CAs can initiate a call message, i.e., can call other contracts and contracts' functions.
- CAs can transfer ether.
- CAs contain a key-value store.
- CAs' addresses are generated when they are deployed. This address of the contract is used to identify its location on the blockchain.

Ethereum Account

- They have access to storage and can manipulate their storage.
- Also, CAs can maintain their permanent states and can call other contracts
- A CA can only be created from an EOA or from an already existing CA (contract).
- CAs cannot start transaction messages.
- CAs can initiate a call message, i.e., can call other contracts and contracts' functions.
- CAs can transfer ether.
- CAs contain a key-value store.
- CAs' addresses are generated when they are deployed. This address of the contract is used to identify its location on the blockchain.

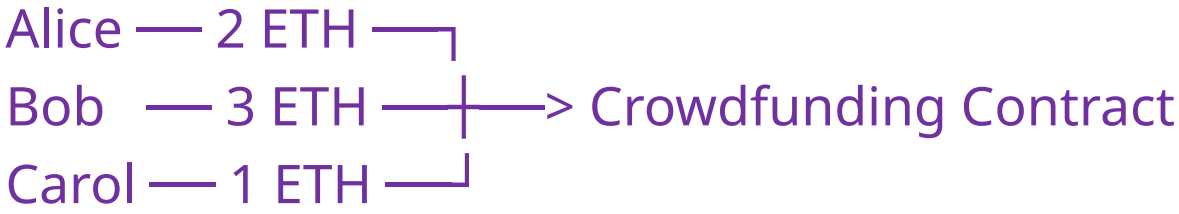
Uses of ETH in Contract Account

A Contract Account (CA) can have an ETH balance, and that ETH can be very useful. The key point is that the ETH is controlled according to the smart contract's code and rules, rather than by a private key.

Why does a Contract Account need ETH?

A smart contract can receive, hold, and transfer ETH.

For example, imagine a crowdfunding smart contract:



Contract Balance = 6 ETH

The contract can later distribute that 6 ETH according to its programmed rules.

A smart contract can:
Store data
Change its stored data
Receive ETH
Transfer ETH
Execute code
Call other smart contracts

Uses of ETH in Contract Account

❑ Holding ETH

A contract can act like an **automated digital vault**.

For example:

User

| Send 5 ETH

▼

Smart Contract

└── Balance = 5 ETH

The contract can hold that ETH until a particular condition is satisfied.

Real-life analogy

Think of an **escrow account**.

Suppose you want to buy a house. You deposit money into an escrow account, and the money is released when the agreed conditions are met.

A smart contract can perform a similar function:

"Hold the ETH until condition X is satisfied."

Uses of ETH in Contract Account

❑ Paying Users

A contract can transfer ETH to users when its rules allow it.

For example, a lottery contract:

Players

| Entry fees

▼

Lottery Contract

| 10 ETH prize

▼

Winner

Suppose:
100 people each pay 0.1 ETH
Contract receives 10 ETH
One winner is entitled to the prize
The contract can transfer the prize according to its programmed logic.

Uses of ETH in Contract Account

❑ DeFi Applications

ETH held by smart contracts is heavily used in **DeFi**.

For example, a lending contract might receive ETH as collateral.

Alice



Lending Contract

- Holds ETH as collateral
- Allows borrowing

Alice may be allowed to borrow another asset against that ETH.

The contract keeps the collateral according to its rules.

Uses of ETH in Contract Account

❑ Escrow

Suppose Alice wants to buy a product from Bob.

Instead of Alice sending ETH directly to Bob:

Alice ————————> Bob

they could use a smart contract:



The contract temporarily holds the ETH.

If the conditions are satisfied, it releases the money to Bob.

Uses of ETH in Contract Account

❑ Paying Rewards

A contract can hold ETH and distribute it as rewards.

For example, a staking/reward application could work conceptually like:

Users



Deposit assets

Smart Contract



Calculate rewards

Users receive ETH

The contract's ETH balance can therefore serve as a source of rewards, depending on the application's design.

Uses of ETH in Contract Account

Can the Contract Use Its ETH to Pay Gas?

This is a **very important distinction**.

No—not directly.

A contract's ETH balance is not automatically used to pay the gas fee for transactions that users send to it.

For example:

Alice's EOA

| Transaction + gas

▼

Smart Contract

The transaction's gas costs are paid according to Ethereum's transaction-fee mechanism by the transaction sender.

So, if Alice calls a contract, Alice's transaction does not normally take the gas fee from the contract's ETH balance.

Uses of ETH in Contract Account

❑ A Complete Example: Crowdfunding

Suppose a crowdfunding contract has a target of **100 ETH**.

Users contribute:

Alice → 20 ETH

Bob → 30 ETH

Carol → 50 ETH

Total → 100 ETH

The contract now has:

Contract balance = 100 ETH

Uses of ETH in Contract Account

❑ A Complete Example: Crowdfunding

The smart contract could be programmed:

If the target reaches 100 ETH, release the funds to the project owner.

Then:

100 ETH



Crowdfunding Contract



Project Owner

If the target isn't reached by the deadline, the contract could instead allow contributors to withdraw their deposits, depending on the contract's design.

Uses of ETH in Contract Account

❑ A Complete Example: Crowdfunding

The smart contract could be programmed:

If the target reaches 100 ETH, release the funds to the project owner.

Then:

100 ETH



Crowdfunding Contract



Project Owner

If the target isn't reached by the deadline, the contract could instead allow contributors to withdraw their deposits, depending on the contract's design.

What is an Ethereum Transaction?

An **Ethereum transaction** is a **signed instruction sent from one Ethereum account to another** that requests the Ethereum network to perform some action.

We can use a transaction to:

- Send **ETH** to another account
- Call a **smart contract function**
- Deploy a **smart contract**
- Transfer tokens such as ERC-20 tokens through a smart contract

What is an Ethereum Transaction?

Simple Transaction Example

Suppose **Alice** wants to send **1 ETH** to **Bob**.

The transaction might conceptually contain:

From: Alice's address

To: Bob's address

Value: 1 ETH

Gas Limit: 21,000

Gas Fee: determined by gas price/fee settings

Nonce: 5

Signature: Alice's digital signature

What is an Ethereum Transaction?

Alice signs the transaction using her **private key** and broadcasts it to the Ethereum network.

The Ethereum network then:

- **Receives the transaction**
- **Validates the digital signature**
- **Checks Alice's nonce and balance**
- **Executes the transaction**
- **Charges gas**
- **Includes the transaction in a block**
- **Updates the Ethereum state**

After successful execution, Bob's balance increases by **1 ETH**, while Alice's balance decreases by **1 ETH plus the transaction fee**.

Legacy Ethereum Transaction

In the Ethereum protocol, a **Legacy Transaction** specifically means the **original Ethereum transaction format**, now commonly called **Type 0**.

There are also three standard legacy transaction types based on the output they produce:

- Simple transactions: This is the standard transaction used for payments. It transfers ether (funds) between EOAs.
- Message call transactions: This transaction is used to execute smart contracts, i.e., used to invoke methods in the deployed smart contracts.
- Contract creation transactions: These transactions result in the creation of a new contract account, i.e., a smart contract—an account with the associated code.

Legacy Ethereum Transaction

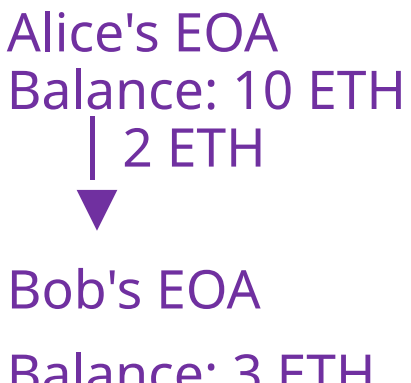
In the Ethereum protocol, a **Legacy Transaction** specifically means the **original Ethereum transaction format**, now commonly called **Type 0**.

There are also three standard legacy transaction types based on the output they produce:

- Simple transactions: This is the standard transaction used for payments. It transfers ether (funds) between EOAs.

Example

Suppose Alice has an Ethereum account and wants to send **2 ETH** to Bob.



Legacy Ethereum Transaction

Alice creates a transaction containing information such as:

From: Alice

To: Bob

Value: 2 ETH

Data: Empty

Gas Limit: 21,000

Gas Price: 30 Gwei

Nonce: 5

Alice then signs the transaction using her **private key**.

Legacy Ethereum Transaction

What happens?

The Ethereum network validates the transaction and, if everything is valid:

Before:

Alice = 10 ETH

Bob = 3 ETH



After:

Alice = 7.99937 ETH

Bob = 5 ETH

The small difference is because Alice also pays the transaction fee.

If:

Gas used = 21,000

Gas price = 30 Gwei

then:

Fee = 21,000 × 30 Gwei

= 0.00063 ETH

Therefore:

Alice:

10 - 2 - 0.00063

= 7.99937 ETH

Bob:

3 + 2

= 5 ETH

Legacy Ethereum Transaction

- Message call transactions: This transaction is used to execute smart contracts, i.e., used to invoke methods in the deployed smart contracts.

Here, the transaction is sent to a **contract account**, rather than to another EOA.

Example: Token Transfer

Suppose Alice owns 100 USDT and wants to send 20 USDT to Bob.

USDT is controlled by a smart contract.

Alice doesn't simply send 20 USDT directly to Bob's account.

Instead, she calls a function of the USDT smart contract:

transfer(Bob, 20)

The transaction might contain:

From: Alice

To: USDT Contract

Value: 0 ETH

Data: transfer(Bob, 20)

Gas Limit: 100,000

Legacy Ethereum Transaction

- Message call transactions: This transaction is used to execute smart contracts, i.e., used to invoke methods in the deployed smart contracts.

Here, the transaction is sent to a **contract account**, rather than to another EOA.

Example: Token Transfer

Suppose Alice owns 100 USDT and wants to send 20 USDT to Bob.

USDT is controlled by a smart contract.

Alice doesn't simply send 20 USDT directly to Bob's account.

Instead, she calls a function of the USDT smart contract:

transfer(Bob, 20)

The transaction might contain:

From: Alice

To: USDT Contract

Value: 0 ETH

Data: transfer(Bob, 20)

Gas Limit: 100,000

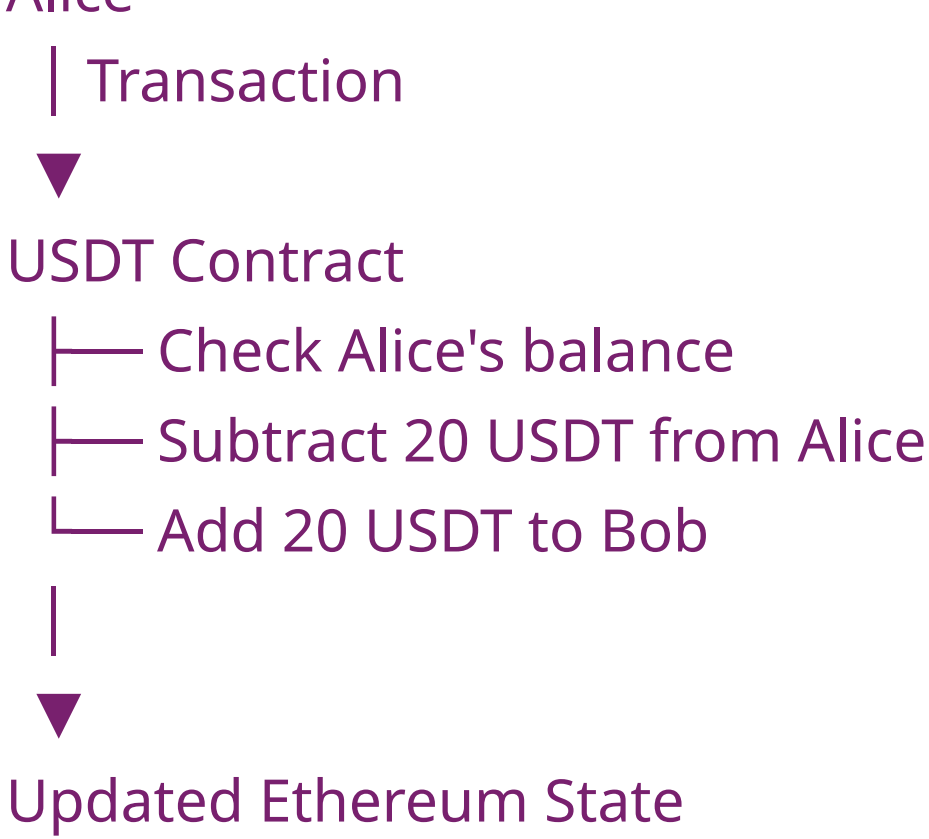
The actual Data field contains **ABI-encoded binary data**, not literally the text transfer(Bob,20).

Legacy Ethereum Transaction

What happens inside Ethereum?

The EVM executes the smart contract's code.

Conceptually:



After execution:

Alice: 100 USDT → 80 USDT

Bob: 0 USDT → 20 USDT

Legacy Ethereum Transaction

Contract Creation Transaction

A contract creation transaction is used to deploy a new smart contract onto Ethereum.

Unlike a message call, there is no existing contract to call.

The transaction contains the contract creation bytecode.

Example

Suppose Alice wants to create a simple storage contract:

```
</>solidity
contract Storage {
    uint256 public value;
    function setValue(uint256 _value) public {
        value = _value;
    }
```

Legacy Ethereum Transaction

Contract Creation Transaction

A contract creation transaction is used to deploy a new smart contract onto Ethereum.

Unlike a message call, there is no existing contract to call.

The transaction contains the contract creation bytecode.

Example

Suppose Alice wants to create a simple storage contract:

```
</>solidity
contract Storage {
    uint256 public value;
    function setValue(uint256 _value) public {
        value = _value;
    }
```

Alice compiles this Solidity program.

The compiler produces **EVM bytecode**.
Alice then creates a contract creation transaction.
Conceptually:

From: Alice
To: EMPTY
Value: 0 ETH
Data: Contract Creation Bytecode
Gas Limit: 500,000